Reliable Machine Learning with Unreliable Black Boxes **Spring 2026**

Lecture 8: Algorithms with Predictions Day 1

Tuesday, April 28, 2026

Instructors: A. Vijayaraghavan and V. Srinivas Scribe: Bob Guo

Overview

This lecture begins our treatment of *algorithms with predictions* (also called *learning-augmented algorithms*). The setting is as follows: an algorithm receives, in addition to its usual input, a black-box "prediction" supplied by some external predictor (typically a machine-learning model). The prediction is not assumed to be accurate. Our goal is to design algorithms that:

- exploit the prediction when it is good, and
- degrade gracefully (ideally to the best worst-case algorithm) when it is bad.

We now illustrate the framework on *searching in a sorted array* as a warm-up.

1 Warm-up: search with predictions

Recall the standard problem of searching for a key in a sorted array of n elements. Binary search solves it using $\Theta(\log n)$ comparisons in the worst case, which is also optimal in terms of number of comparisons.

Now suppose we are also given a *predicted location* $p \in \{1, 2, \ldots, n\}$: a guess for the index at which our key resides. We have no guarantee on the quality of p. Define the prediction error

$$\mathrm{err}(p) := |\, p - p^* \,|,$$

where p^* is the true index of the key. We measure the quality of the prediction by how many array entries it is off by.

Theorem 1. *There is a comparison-based search algorithm that, given a sorted array of length n, a query, and a predicted location p, finds the query (or reports its absence) using*

$$O\big(\log \mathrm{err}(p) + 1\big)$$

comparisons in the worst case.

Proof. The algorithm has two phases.

Phase 1 (exponential expansion). Probe the array at index p. If we have not found the key, probe successively at indices

$$p \pm 1,\ p \pm 2,\ p \pm 4,\ p \pm 8,\ \ldots,\ p \pm 2^j,\ \ldots$$

until we identify a window $[\,p - 2^j,\, p + 2^j\,]$ that brackets the key (i.e., the comparisons at the endpoints rule out the rest of the array, by the sortedness of the array).

After at most $j = \lceil \log_2 \mathrm{err}(p) \rceil + 1$ rounds, the window contains the true index p^*, since $|p^* - p| = \mathrm{err}(p) \le 2^j$. Phase 1 therefore uses $O(\log \mathrm{err}(p))$ comparisons, and produces a window of width $2 \cdot 2^j \le 4 \cdot \mathrm{err}(p)$.

Phase 2 (binary search). Run binary search inside the bracketing window. Since the window has width $O(\mathrm{err}(p))$, this takes $O(\log \mathrm{err}(p))$ comparisons.

The total cost is $O(\log \mathrm{err}(p))$, as claimed. □

A few comments:

- When $\mathrm{err}(p) = 0$, the algorithm returns the answer in $O(1)$ time, matching the cost of just trusting the prediction.
- When $\mathrm{err}(p) = \Theta(n)$, the bound becomes $O(\log n)$, matching plain binary search up to constants.
- More generally, the running time interpolates smoothly between these two extremes as a function of $\mathrm{err}(p)$ alone.

2 The algorithms-with-predictions paradigm

Let $\mathcal{A}$ be a learning-augmented algorithm for some problem, and let η denote some measure of prediction error (e.g., $\mathrm{err}(p)$ above). We compare $\mathcal{A}$'s cost to OPT, the cost of the best offline solution.

Definition 2 (Goals of the paradigm)**.** A learning-augmented algorithm $\mathcal{A}$ should satisfy, ideally, all three of the following.

1. **Robustness.** Even when the prediction is adversarial, $\mathcal{A}$ should perform almost as well as the best prediction-oblivious algorithm. (In other words, $\mathcal{A}$'s competitive ratio should be no worse than that of the best classical algorithm.)
2. **Consistency.** When the prediction is perfect, $\mathcal{A}$ should perform almost as well as "blindly trusting the prediction." (Often much better than what is achievable without predictions.)
3. **Graceful degradation (smoothness).** The performance of $\mathcal{A}$ should degrade as a smooth function of η, interpolating between the consistency and robustness regimes.

Note that the search algorithm of Theorem 1 achieves all three goals.

3 Online caching

We now turn to a main example, the online caching (or *paging*) problem. We will analyze caching *without* predictions in this lecture, setting up the next lecture where predictions enter the picture.

3.1 Problem definition

There is a universe U of m pages and a cache of size $k < m$. A request sequence $\sigma = e_1, e_2, \ldots, e_T$ is revealed online, one request at a time. On request e_t:

- If e_t is in the cache, the algorithm pays cost 0 (a *hit*).
- If e_t is not in the cache, the algorithm pays cost 1 (a *miss*); it must then evict some page from the cache and load e_t in its place.

The total cost of an algorithm is the number of misses it incurs over the whole sequence.

3.2 Competitive analysis

We measure the quality of an online caching algorithm by comparing it against the offline optimum OPT, which knows the entire request sequence in advance and chooses the best possible eviction strategy in hindsight.

Definition 3 (Competitive ratio)**.** An online algorithm ALG is *c-competitive* if, for every request sequence σ,

$$\mathbb{E}[\,\mathrm{ALG}(\sigma)\,] \le c \cdot \mathrm{OPT}(\sigma) + O(1).$$

The smallest such c is called the *competitive ratio* of ALG.

A classical fact that no deterministic online caching algorithm can achieve competitive ratio better than k, and that the classical algorithms LRU (least recently used) and FIFO (first-in-first-out) both achieve k. We will now show that randomization helps: the *Marking* algorithm achieves a competitive ratio of $O(\log k)$.

3.3 The Marking algorithm

The Marking algorithm [Fiat, Karp, Luby, McGeoch, Sleator, and Young, 1991] maintains, in addition to the cache contents, a binary "marked / unmarked" label on each cache entry.

Algorithm (Marking). On request e:

- If e is in the cache, mark e.
- Otherwise (a miss):
  - If every page in the cache is currently marked, unmark them all.
  - Evict a uniformly random unmarked page; insert e and mark it.

The act of unmarking everyone partitions time into *phases*.

Definition 4 (Phases, clean and stale items)**.** A *phase* is an interval of time between two consecutive "unmark all" events (or from the start of the sequence to the first unmark).

For phase i, an item is *clean* if it is requested during phase i but *not* during phase $i-1$. Otherwise (i.e., requested in both phase i and phase $i-1$) it is *stale*. Let ℓ_i denote the number of clean items in phase i.

A few easy facts that we record without proof:

- Marked items are never evicted, so once an item is marked in a phase, it stays in cache for the rest of the phase.
- Every requested item gets marked (whether the request was a hit or a miss). Hence the set of items requested in phase i equals the set of items in cache at the end of phase i.
- In particular, exactly k distinct items are requested in each phase: phases end precisely when the cache is fully marked and a new (clean) item is requested.

Theorem 5 (Marking is $O(\log k)$-competitive [Fiat, Karp, Luby, McGeoch, Sleator, and Young, 1991])**.** *On any request sequence, the expected number of misses incurred by Marking is at most* $O(\log k) \cdot \mathrm{OPT} + O(k \log k)$.

The proof has two pieces: a lower bound on OPT in terms of the ℓ_i's, and an upper bound on $\mathbb{E}[\mathrm{ALG}]$ also in terms of the ℓ_i's. We treat them in turn.

Lower bound on OPT**.**

Fix a phase i. Let $A_{\text{init}}, A_{\text{fin}}$ denote the contents of the algorithm's cache at the start and the end of phase i, and similarly $O_{\text{init}}, O_{\text{fin}}$ for OPT's cache. Define

$$d_{\text{init}} := |O_{\text{init}} \setminus A_{\text{init}}|, \qquad d_{\text{fin}} := |O_{\text{fin}} \setminus A_{\text{fin}}|.$$

Both quantities lie in $[0, k]$ and measure how much OPT's cache disagrees with ALG's cache at the two endpoints of phase i.

Lemma 6. *The number of misses* OPT *incurs during phase i satisfies*

$$\text{OPT } \textit{misses in phase } i \;\geq\; \ell_i - d_{\text{init}} \qquad \textit{and} \qquad \text{OPT } \textit{misses in phase } i \;\geq\; d_{\text{fin}}.$$

Consequently,

$$\text{OPT } \textit{misses in phase } i \;\geq\; \tfrac{1}{2}\big(\ell_i - d_{\text{init}} + d_{\text{fin}}\big).$$

Proof. First inequality. By definition, the ℓ_i clean items of phase i are not in A_{init}. Among these ℓ_i items, at most $d_{\text{init}} = |O_{\text{init}} \setminus A_{\text{init}}|$ can already be in OPT's cache. So at least $\ell_i - d_{\text{init}}$ of them are not in O_{init}, and each will cause a miss for OPT when first requested in phase i.

Second inequality. Recall A_{fin} equals the set of items requested in phase i. Since $|O_{\text{fin}}| = |A_{\text{fin}}| = k$, we have

$$|A_{\text{fin}} \setminus O_{\text{init}}| = k - |A_{\text{fin}} \cap O_{\text{init}}| = |O_{\text{init}} \setminus A_{\text{fin}}|.$$

Now, the d_{fin} items in $O_{\text{fin}} \setminus A_{\text{fin}}$ were not requested during phase i (since they are not in A_{fin}) but are in OPT's cache at the end of the phase; since OPT inserts an item into its cache only upon requesting it, these d_{fin} items must have been in O_{init} as well. Hence $|O_{\text{init}} \setminus A_{\text{fin}}| \geq d_{\text{fin}}$, and therefore

$$|A_{\text{fin}} \setminus O_{\text{init}}| = |O_{\text{init}} \setminus A_{\text{fin}}| \geq d_{\text{fin}}.$$

Each of the items in $A_{\text{fin}} \setminus O_{\text{init}}$ is requested in phase i but is not in OPT's cache at the start, so OPT incurs a miss on its first request. The second inequality follows.

Average. Adding the two inequalities and dividing by 2 yields the third claim. □

Summing the per-phase bound across all phases and using the telescoping identity $d^i_{\text{fin}} = d^{i+1}_{\text{init}}$ (the algorithm and OPT do not change between phases), we obtain

$$\text{OPT} \;\geq\; \sum_i \tfrac{1}{2}\big(\ell_i - d^i_{\text{init}} + d^i_{\text{fin}}\big) \;\geq\; \tfrac{1}{2}\sum_i \ell_i \;-\; O(k). \tag{1}$$

Upper bound on $\mathbb{E}[\text{ALG}]$.

Fix a phase i once more. The ℓ_i clean items contribute ℓ_i certain misses, since they are not in cache at the start of the phase. We must therefore bound the expected number of *stale* misses—that is, requests in phase i to items that were in cache at phase start, but were evicted between phase start and the request.

Eviction chains. Imagine the misses of phase i as triggering chains of evictions. The j-th clean miss starts the j-th chain: it evicts a uniformly random unmarked page, say a. If a is later requested in phase i, that request is itself a (stale) miss, which evicts another uniformly random unmarked page, say b, and so on:

$$(\text{clean item}) \xrightarrow{\text{evicts}} (a) \xrightarrow{\text{evicts}} (b) \xrightarrow{\text{evicts}} \cdots$$

The chain terminates when an evicted page is never re-requested in the phase. Two key observations:

leftmirgin=* Each chain starts with a clean miss, so there are exactly ℓ_i chains in phase i.

leftmiirgiin=* Every miss in phase i belongs to exactly one chain (the chain initiated by the most recent clean miss whose "ripple" reached this request).

Hence the total number of misses in phase i is the sum of the chain lengths.

Lemma 7. *Let L denote the length of a single chain (starting from a fixed clean miss, in a fixed phase). Then $\mathbb{E}[L] = O(\log k)$.*

Proof sketch. At each step of the chain, the evicted page is chosen uniformly at random from the currently unmarked pages in the cache. There are at most k unmarked pages at any time, and the number of unmarked pages decreases by one each time a page is marked (in particular, after each step of the chain). Conditioning on the random choices, one can show that the probability

the chain extends by one more step, given that it has reached length j, is at most a quantity that decays like $1/(k-j+1)$. Summing the expected contributions yields

$$\mathbb{E}[L] \;\le\; 1 + \tfrac{1}{k} + \tfrac{1}{k-1} + \cdots + \tfrac{1}{1} \;=\; H_k \;=\; O(\log k),$$

where H_k is the k-th harmonic number. (See [Fiat, Karp, Luby, McGeoch, Sleator, and Young, 1991] for the full argument.) □

Combining the two observations and Lemma 7:

$$\mathbb{E}[\,\text{ALG misses in phase } i\,] = \sum_{j=1}^{\ell_i} \mathbb{E}[\text{length of } j\text{-th chain}] \le \ell_i \cdot O(\log k).$$

Summing across phases,

$$\mathbb{E}[\,\text{ALG}\,] \;\le\; O(\log k) \cdot \sum_i \ell_i. \tag{2}$$

Putting it together.

Combining (1) and (2),

$$\mathbb{E}[\,\text{ALG}\,] \;\le\; O(\log k) \cdot \sum_i \ell_i \;\le\; O(\log k) \cdot \big(2\text{OPT} + O(k)\big) \;=\; O(\log k) \cdot \text{OPT} + O(k \log k),$$

which proves Theorem 5. □

3.4 Looking ahead

The Marking algorithm achieves an $O(\log k)$ competitive ratio against an oblivious adversary, and a matching lower bound of $\Omega(\log k)$ is known for any randomized online caching algorithm. So in the standard online setting, $\Theta(\log k)$ is the right answer.

In the prediction model of Lykouris and Vassilvitskii [Lykouris and Vassilvitskii, 2021], when each requested page arrives, it is annotated with a (possibly inaccurate) prediction of the time of its next request. Bélády's offline rule says that an offline algorithm with perfect knowledge of next-request times should always evict the page whose next request is furthest in the future. We can combine this rule with the Marking algorithm to obtain an algorithm that is simultaneously $O(\log k)$-competitive (robustness, matching the prediction-oblivious benchmark of Theorem 5) and consistent up to a small constant factor when the predictions are exact, with smooth degradation in between as a function of a natural error measure on the predictions.

References

Amos Fiat, Richard M Karp, Michael Luby, Lyle A McGeoch, Daniel D Sleator, and Neal E Young. Competitive paging algorithms. *Journal of Algorithms*, 12(4):685–699, 1991. ISSN 0196-6774. doi: https://doi.org/10.1016/0196-6774(91)90041-V. URL https://www.sciencedirect.com/science/article/pii/019667749190041V.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *J. ACM*, 68(4), July 2021. ISSN 0004-5411. doi: 10.1145/3447579. URL https://arxiv.org/abs/1802.05399.